

MASTER'S THESIS DEFENSE

Enhancing Stability in Direct Training of Spiking Neural Networks

Membrane Potential Initialization & Threshold-Robust Surrogate Gradients

Hyunho Kook (국 현 호)

Advisor: Prof. Eunhyeok Park

Committee: Prof. Jungseul Ok · Prof. Jae-Joon Kim

Why Spiking Neural Networks?

DNNs are powerful but expensive

Training and serving large models routinely consumes megawatt-scale power; even edge inference is hard under tight energy budgets.

SNNs: an event-driven alternative

Information flows as discrete spikes over time. Paired with neuromorphic hardware, computation skips inactive neurons — orders of magnitude lower energy.

But direct training is unstable

Recent direct-training methods narrowed the accuracy gap, yet two persistent instabilities still hold SNNs back.

THIS THESIS

Two instabilities, two principled fixes.

① Temporal Covariate Shift

in the membrane potential — invisible to existing remedies.

② Gradient pathology

when the firing threshold becomes a learnable parameter — never explicitly characterized before.

Spiking Neurons in 60 Seconds

Three discrete-time updates, plus one trick to get gradients through them

Leaky Integrate-and-Fire neuron

Integrate

$$M[t] = (1 - 1/\tau) \cdot U[t-1] + (1/\tau) \cdot I_{\text{in}}[t]$$

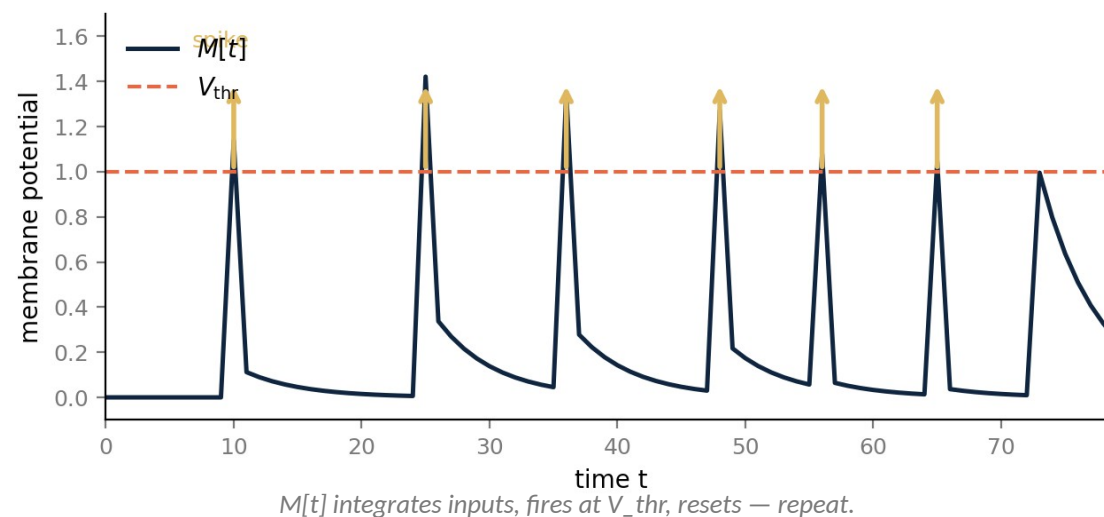
Fire

$$S[t] = H(M[t] - V_{\text{thr}}) \in \{0, 1\}$$

Reset

$$U[t] = M[t] - V_{\text{thr}} \cdot S[t] \quad (\text{soft reset})$$

Two learnable parameters: τ (leakage) · V_{thr} (threshold).



Heaviside H is not differentiable

Surrogate gradient: backward pass uses smooth $f'(x) \approx H'(x)$.

Everything in this thesis turns on $M[t]$ and V_{thr} · TCS at $M[t]$ · pathology at V_{thr}

Diagnosis ①: TCS Lives in the Membrane Potential

What everyone notices

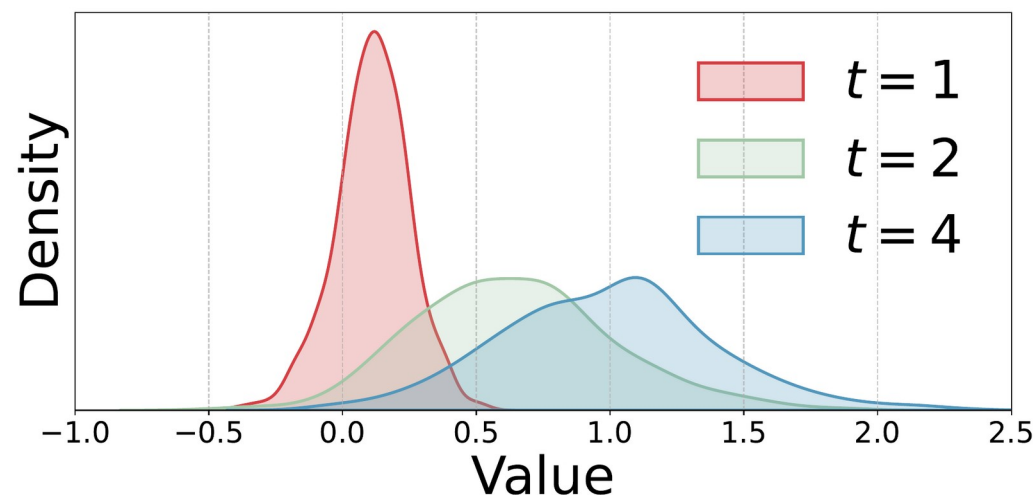
Activation distributions drift across timesteps. Batch-norm statistics estimated at one timestep no longer fit at another — accuracy drops.

What existing fixes do

TEBN, TAB add per-timestep BN parameters.
More compute, more parameters — and they don't reach the cause.

Our claim

TCS originates one level deeper — in $M[t]$ itself.



$M[t]$ under tdbn — distribution clearly drifts as t grows

Diagnosis ②: Trainable V_{thr} Breaks the Gradient

PLIF, LTMD, DIET-SNN all let V_{thr} learn — but the surrogate gradient is scale-sensitive

$V_{thr} \ll 1$ (small threshold)

- ▶ AS-SG floods — fixed window covers almost the entire distribution
- ▶ RS-SG explodes — the $1/V_{thr}$ factor blows the magnitude up
- ▶ Real example: at $V_{thr} = 0.1$, RS-SG diverges to NaN within 100 epochs

$V_{thr} \gg 1$ (large threshold)

- ▶ AS-SG starves — fixed window covers a tiny tail of the distribution
- ▶ RS-SG vanishes — the same $1/V_{thr}$ factor shrinks the magnitude
- ▶ Real example: at $V_{thr} = 2.0$, AS-SG hits 0% active neurons by epoch 100

These are not corner cases.

Trainable V_{thr} in practice drifts well below 1 in early layers and above 2 in deep ones — exactly the regimes where AS-SG and RS-SG both fail. We'll quantify in Part II.

Why Does the Membrane Potential Drift?

Because it's a Markov chain pulled toward a unique distribution

The recurrence is a Markov chain

$U[t+1] = f(U[t], I_{in}[t+1])$ — i.i.d. $\Rightarrow$ Markov chain

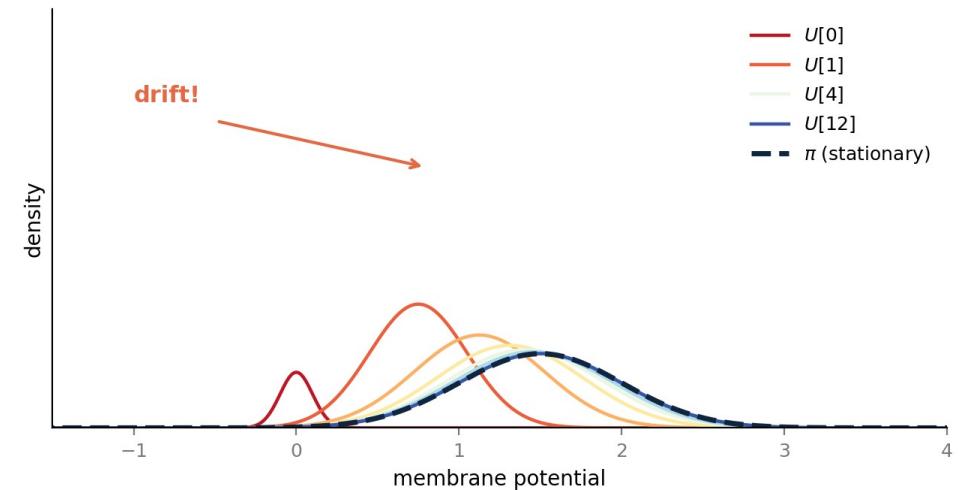
Theorem (Doebelin minorization)

Under (i.i.d. inputs) + (bounded $U[t]$),
the chain has a unique stationary π . Moreover,

$$\|P^n(x, \cdot) - \pi(\cdot)\|_{TV} \leq C \cdot (1 - \epsilon)^n$$

Implications

- ▶ Initial state $\neq \pi \Rightarrow$ drift is inevitable
- ▶ Standard $U[0] = 0$ is far from π
- ▶ Don't fix BN — fix the initial state



Distribution of $U[t]$ for $t = 0, 1, 2, 3, 4, 5, 8, 12$

Starting from $U[0]=0$ (sharp red), the distribution exponentially approaches the stationary π (dashed navy).

MP-Init — Initialize at the Stationary Mean

$U[T]$ is the closest sample to $\pi \rightarrow$ use a running mean of $E[U[T]]$

Lemma — best constant is the mean

$$\operatorname{argmin}_c E[(\pi - c)^2] = E[\pi]$$

Practical estimator

$U[T]$ is the closest sample to π . Maintain a per-layer running mean

$\mu \leftarrow \beta \cdot \mu + (1 - \beta) \cdot \text{mean}(U[T])$ over training, $\beta = 0.9$.

At every batch, set $U[0] = \mu$.

Algorithm (training-only)

```
init  $\mu_l \leftarrow 0$  for each layer  $l$ 

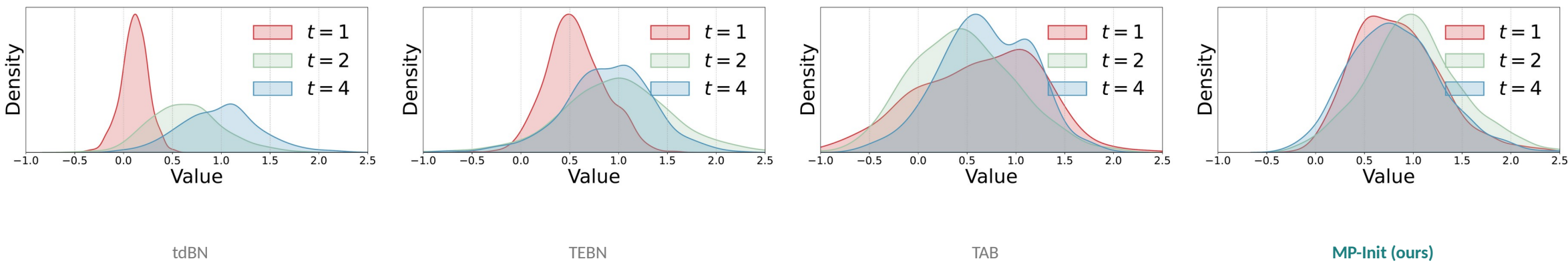
for each batch:
   $U_l[0] \leftarrow \mu_l$ 
  run  $T$  LIF steps as usual
   $m \leftarrow \text{mean}(U_l[T] \text{ of active neurons })$ 
   $\mu_l \leftarrow \beta \cdot \mu_l + (1 - \beta) \cdot m$ 
```

Inference: μ frozen $\rightarrow$ standard LIF dynamics

Zero changes to BN, no per-timestep parameters, full neuromorphic-hardware compatibility.

MP-Init Works — at Both the Distribution and the Metric

Membrane potential distributions across timesteps (ResNet-19 / CIFAR-100)



Consistent accuracy gain on every baseline

Setting	Baseline	+ MP-Init
CIFAR-100 · T=4 · tdBN	75.55	76.09
CIFAR-100 · T=4 · TEBN	75.96	76.45
CIFAR-100 · T=4 · TAB	76.25	77.24
DVS-CIFAR10 · T=10 · tdBN	76.60	77.37

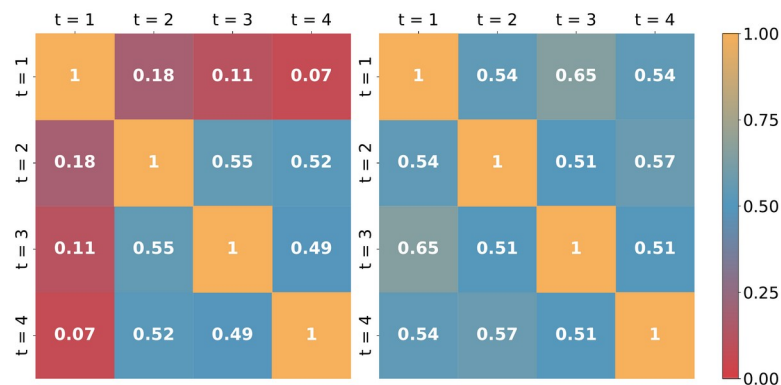
+1.66 %opt at T = 2

Biggest gain in the most latency-critical regime — exactly where SNN energy efficiency matters.

Why MP-Init Works — A Closer Look

Aligning distributions aligns gradients — and improves the firing-rate frontier

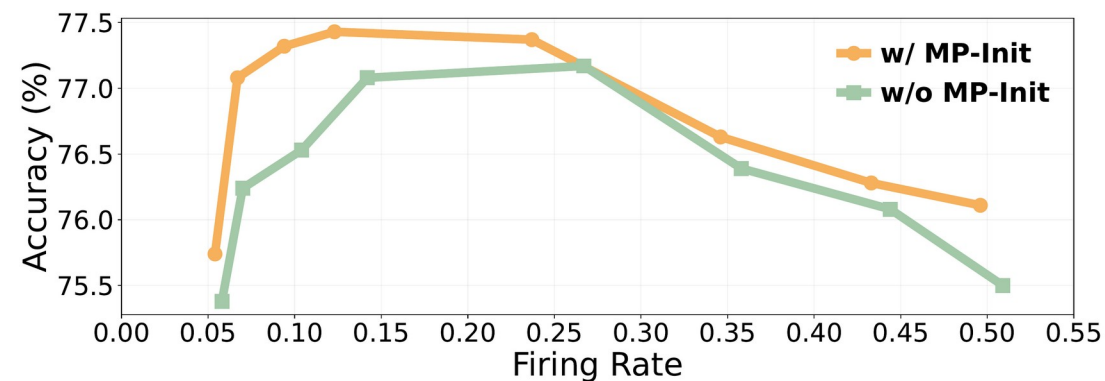
Gradient cosine similarity across timesteps



ResNet-19 • CIFAR-100 • 4 timesteps

- ▶ Without MP-Init: gradients at $t=1$ conflict with later steps (cos 0.07–0.18)
- ▶ With MP-Init: temporal ensemble realigned (cos ≈ 0.5)
- ▶ Stable optimization — not just stable inference

Accuracy vs. firing rate (Pareto frontier)



ResNet-19 • CIFAR-100 with sparsity regularizer

- ▶ MP-Init dominates baseline at every firing rate
- ▶ Even at 6% firing rate, beats all baseline settings
- ▶ Gains come from spiking BETTER, not spiking MORE

Higher accuracy from stabler training — efficiency-preserving improvements, not accuracy-via-spend.

Two Surrogate-Gradient Families

AS-SG and RS-SG — look similar, behave very differently when V_{thr} is trainable

Recap — Surrogate Gradient: replace $H'(x)$ with smooth $f'(x)$. Choice of x differs below.

AS-SG • Absolute-Scale

$$x = M[t] - V_{thr}$$

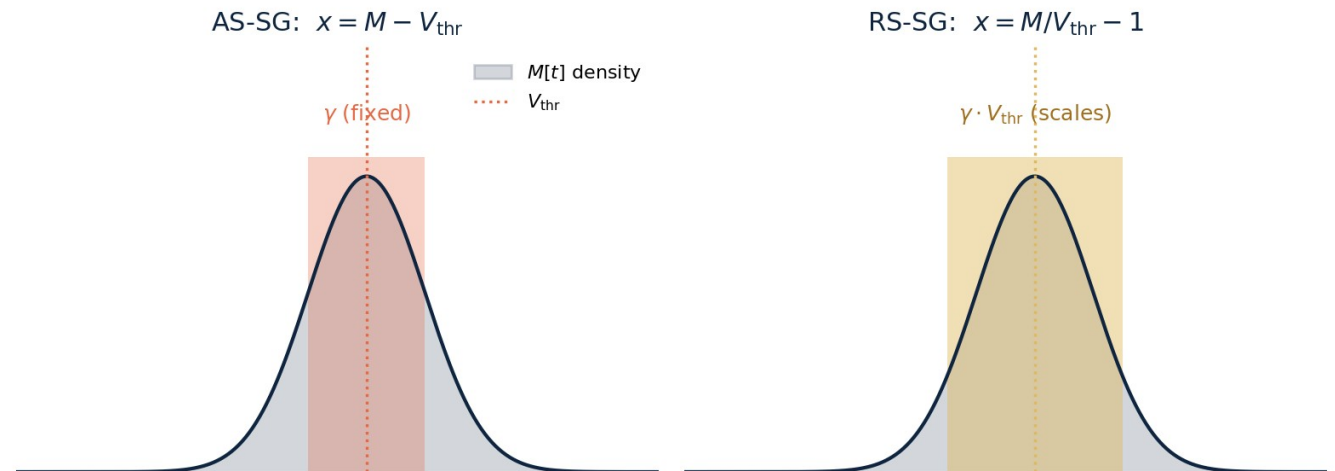
Window of FIXED width γ , centered at V_{thr}

RS-SG • Relative-Scale

$$x = M[t] / V_{thr} - 1$$

Window SCALES with V_{thr} — but magnitude $/ V_{thr}$

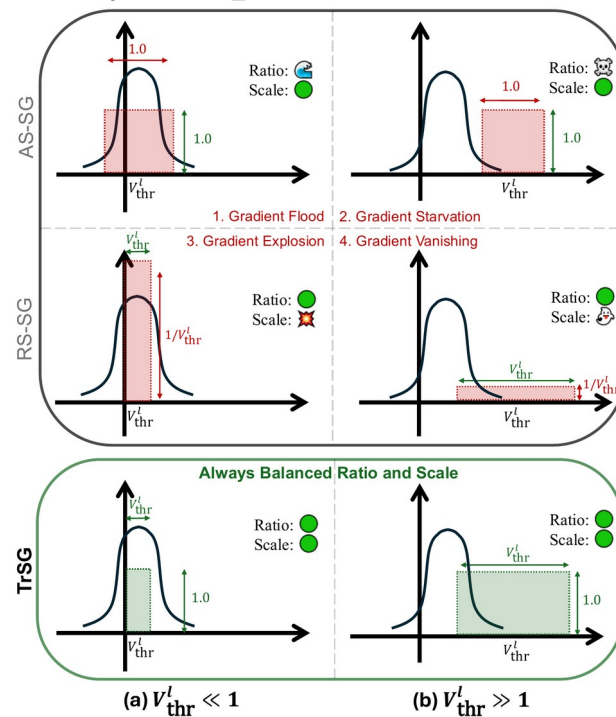
Gradient-window vs membrane distribution



Same when V_{thr} is fixed. Once V_{thr} trains, AS-SG and RS-SG break in opposite ways.

Four Failure Modes — and One Fix

Both window width AND gradient magnitude must scale correctly with V_{thr}



✗ Flooding

$$AS-SG \cdot V_{thr} \ll 1$$

✗ Starvation

$$AS-SG \cdot V_{thr} \gg 1$$

✗ Explosion

$$RS-SG \cdot V_{thr} \ll 1$$

✗ Vanishing

$$RS-SG \cdot V_{thr} \gg 1$$

✓ Balanced

$$TrSG \cdot \text{all regimes}$$

TrSG — One-Line Fix

Multiply the threshold on the forward path; chain rule cancels the bad factor

The recipe

1

Relative-scale window

$$x = M / V_{\text{thr}} - 1$$

2

Multiply on forward

$$O[t] = V_{\text{thr}} \cdot S[t]$$

3

Chain rule cancels

$$\partial O / \partial M = f'(x)$$

Chain-rule cancellation



Window adapts with V_{thr} · gradient magnitude is threshold-invariant.

Inference Stays Binary

TrSG is a training-only trick — neuromorphic hardware never sees it

Training

Forward: $O[t] = V_{thr} \cdot S[t]$ (real-valued — gradient flows cleanly through V_{thr})

At deployment, absorb V_{thr} into the next layer's weights

$W'_{\{l \rightarrow l+1\}} = V_{thr} \cdot W_{\{l \rightarrow l+1\}}$ (one-time rescaling)

Forward at test time: $O[t] = S[t] \in \{0, 1\}$ (standard LIF — back to binary spikes)

Standard LIF operators at inference. Hardware compatibility preserved.

Why TrSG Works — A Stable Gradient

Three stability metrics tell the same story across extreme V_thr

AbsStr

mean |gradient|

low → vanishing, high → exploding

RatioAG

% neurons with active gradient

low → starvation, high → flooding

GradCV

std / mean of |gradient|

high → unstable, hard-to-tune updates

Stress-test metrics at epoch 100 — CIFAR-100, ResNet-19, $\tau=2.0$ frozen

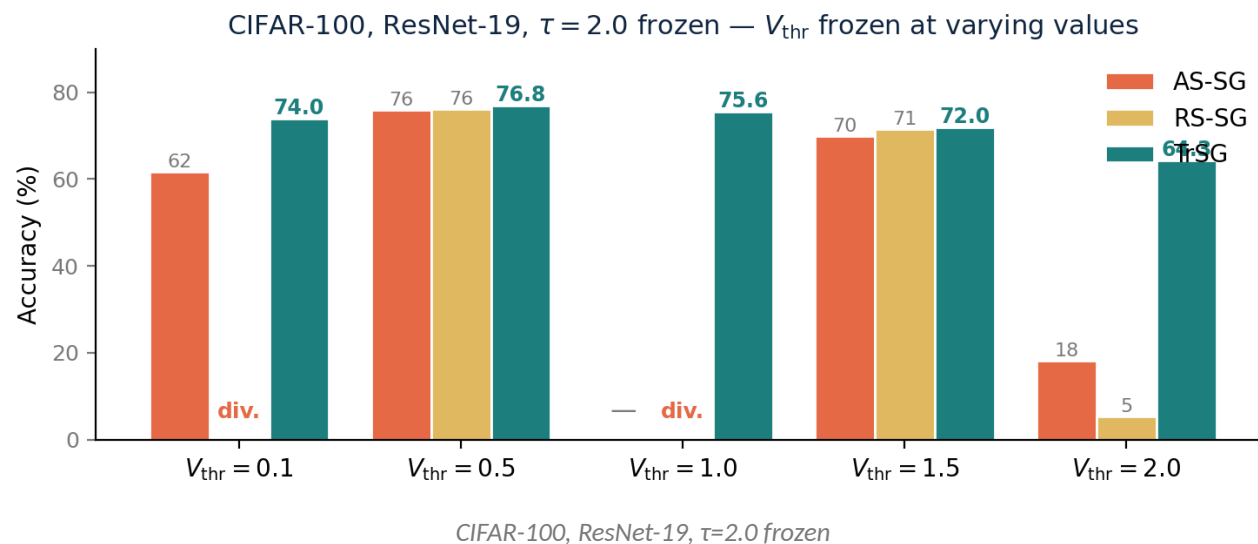
V_thr = 0.1 (small threshold)				V_thr = 2.0 (large threshold)			
SG	AbsStr	RatioAG	GradCV	SG	AbsStr	RatioAG	GradCV
AS-SG	0.0145	90.20%	1.22	AS-SG	0.0000	0.00%	0.000
RS-SG	NaN	0.00%	NaN	RS-SG	0.0021	0.10%	2.25
TrSG	0.0153	21.45%	0.87	TrSG	0.0323	6.90%	0.73

TrSG keeps all three metrics in the healthy range — at every threshold.

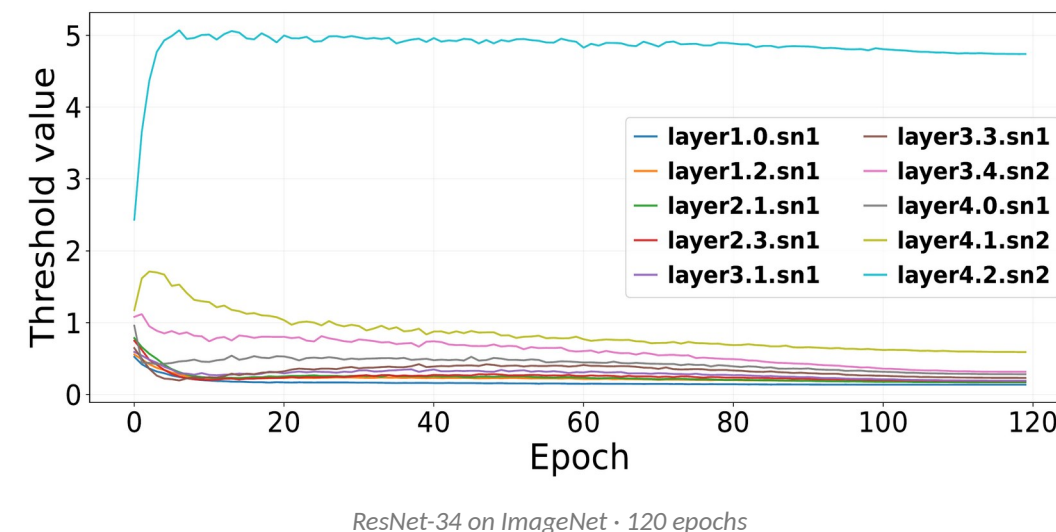
TrSG Robust Across All V_{thr}

Direct accuracy evidence + real trainings actually visit the extreme regimes

Accuracy with V_{thr} frozen at varying values



Trainable V_{thr} trajectories — ImageNet

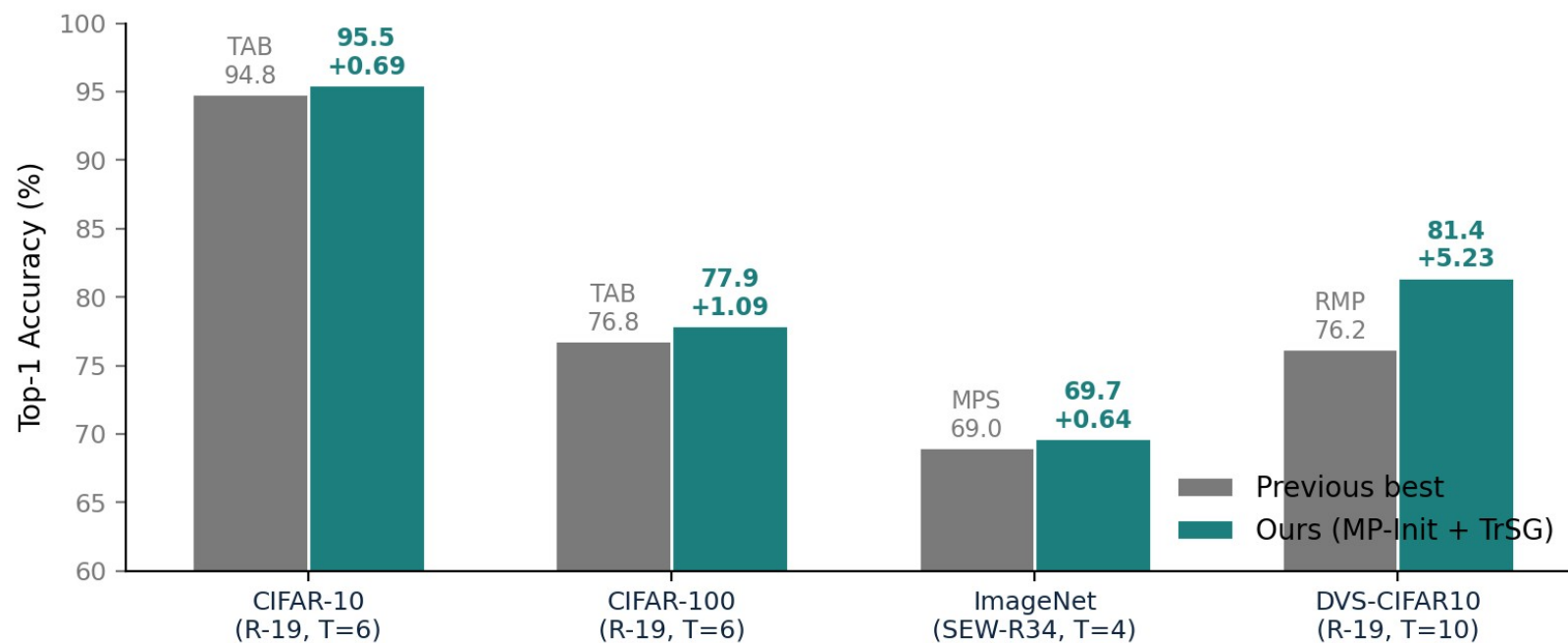


- ▶ Frozen V_{thr} accuracy: TrSG wins at every threshold; AS-SG/RS-SG fail outside $V_{thr} \approx 1$
- ▶ Real trainings drift $V_{thr} \ll 1$ in early layers and $\gg 1$ in deep layers — exactly the failure regimes

Threshold-robustness is a practical necessity, not a theoretical edge case.

Main Results — SOTA Across the Board

Standard LIF neurons • no AutoAugment / Cutout • three trials



+1.09 %pt

CIFAR-100 • beats TAB

+0.64 %pt

ImageNet • beats MPS

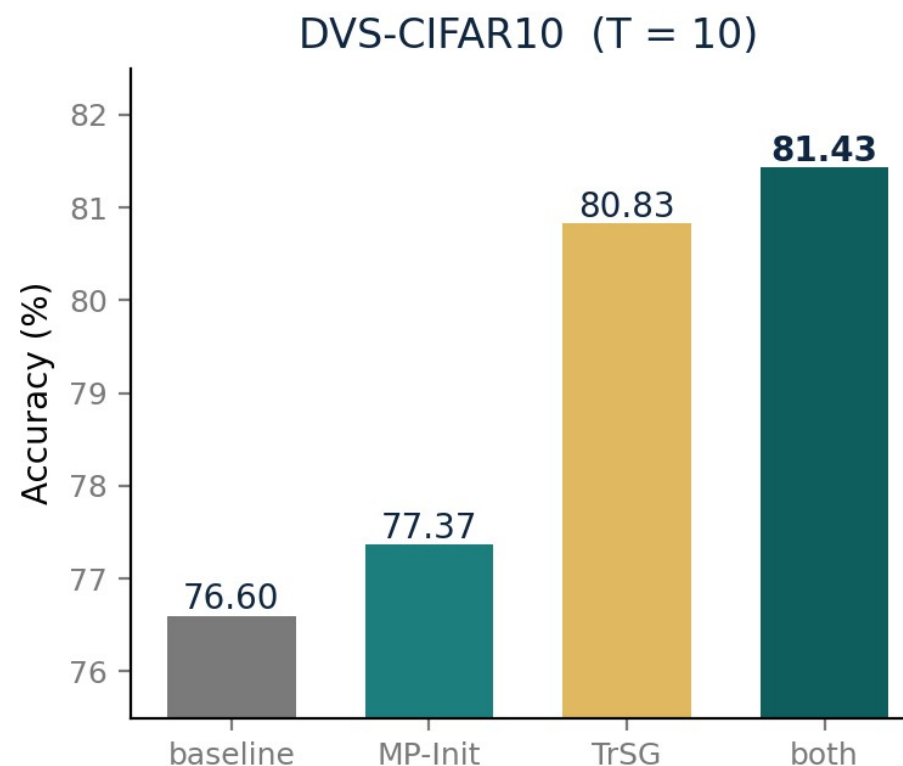
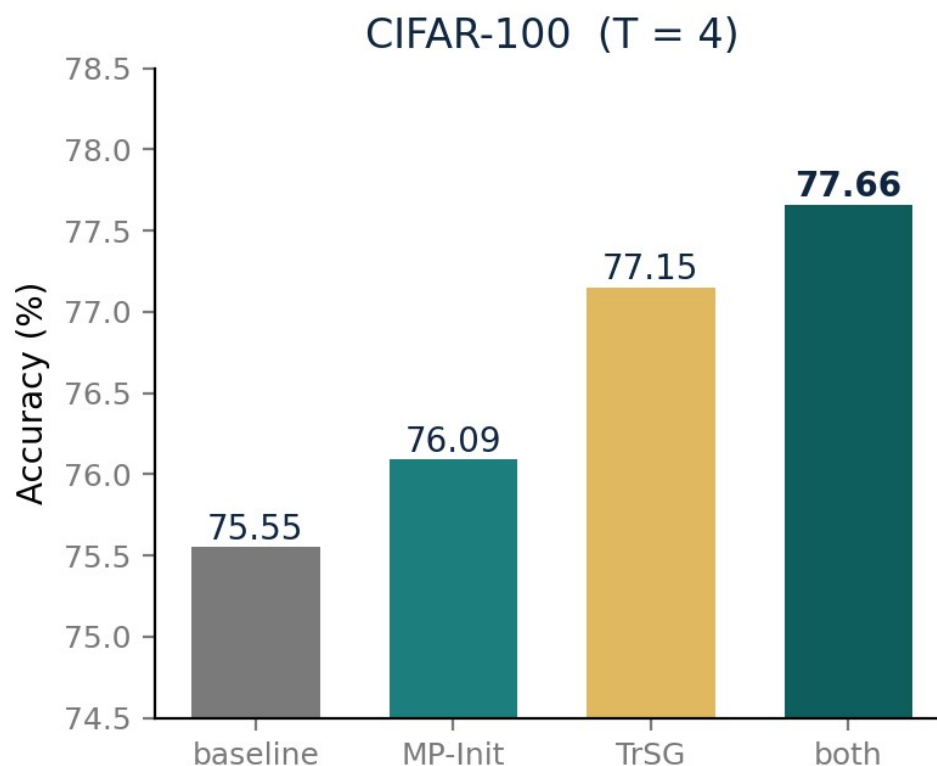
+5.23 %pt

DVS-CIFAR10 • beats RMP

Biggest gain on the most challenging (event-based) dataset — DVS-CIFAR10 +5.23 %pt

Ablation — Independent and Complementary

Each method helps alone; together they compound (especially on event-based data)



Takeaway

MP-Init alone

+0.5 ~ 0.8

TrSG alone

+1.6 ~ 4.2

Both

+2.1 ~ 4.8

The two fixes are orthogonal — diagnoses aim at different components, not the same problem twice.

Beyond Classification

Transformer SNNs • object detection • energy preserved

Transformer SNN (QKFormer, ImageNet, T=4)

Baseline 78.80 → + ours 79.90 (+1.10 %pt)
AS-SG 77.09 → RS-SG 78.70 → TrSG 79.90
Drop-in SG swap on a state-of-the-art Transformer SNN.

Object detection (COCO-2017, EMS-ResNet10, T=3)

mAP@0.5 0.291 → 0.305 (+0.014)
mAP@0.5:0.95 0.138 → 0.147 (+0.009)
First evidence the principles transfer beyond classification.

Energy at matched firing rate (CIFAR-100)

Baseline 1.57 mJ · 75.58 %
+ MP-Init + TrSG 1.67 mJ · 77.68 %
+2.10 %pt at +6 % energy — gains aren't from spiking more.

All four surrogate shapes

Rectangular · Triangular · Arctan · Sigmoid
TrSG remains the best across initial V_{thr} , initial τ ,
and surrogate shape — never tied for second.

No architectural changes, no extra losses, no per-timestep parameters — just drop in.

What This Talk Was About

Two long-standing instabilities of direct SNN training, traced and fixed.

Both fixes are minimal, principled, and preserve standard inference.

MP-Init

TCS originates in the membrane potential — fix the initial state.

Per-layer running mean of $U[T]$.

Zero overhead, standard LIF inference.

TrSG

Threshold-induced gradient pathology — fix it with one forward-path multiplication.

Window adapts, magnitude is invariant.

Standard LIF after training.

Validated

SOTA on CIFAR-10/100, ImageNet, DVS-CIFAR10.

Generalizes to Transformer SNNs (QKFormer +1.10) and detection (COCO +0.014 mAP).

All without architectural changes.

CONCLUSION

**Two instabilities, two principled fixes,
one cohesive story.**

LIMITATIONS

- ▶ Per-layer constant approximates π only at the mean — richer parameterizations are open.
- ▶ Hardware with fixed neuron parameters needs quantization-aware adaptation.
- ▶ i.i.d. assumption empirically supported, but tighter bounds welcome.

FUTURE WORK

- ▶ Hardware-aware adaptation: Loihi, Akida, TrueNorth
- ▶ Beyond LIF: adaptive LIF, Izhikevich, foundation-scale SNNs

Thank you. Questions?

BACKUP MATERIAL

Appendix

Q&A reference: hardware, datasets, prior work

- A1** Can SNNs run on real chips?
- A2** Where SNN efficiency comes from
- A3** Energy: SNN chip vs DNN platforms
- A4** What is an event-based dataset?

- A5** From DVS events to SNN input
- A6** How were SNNs trained before?
- A7** Prior work on TCS & threshold learning

Can SNNs Actually Run on Chips?

Production-grade neuromorphic silicon already exists.

Yes — and at scale.

Several research and commercial chips natively run spiking models. Our methods (MP-Init, TrSG) are training-time techniques that produce the same LIF model these chips already accept — so the trained network ports directly without architectural changes.

Key takeaway

Hardware is here. The bottleneck is training quality at low T.

Chip	Year	Process	Capacity	Energy / power
IBM TrueNorth	2014	65 nm	1 M neur. 256 M syn.	70 mW · 26 pJ/event
Intel Loihi	2018	14 nm	130 K neur. 130 M syn.	23.6 pJ / op
Intel Loihi 2	2021	Intel 4	1 M neur. graded spikes	improved · multi-bit
SpiNNaker 2	2024	22 nm	10 M neur. 152 ARM cores	ARM-based · digital
Tianjic	2019	28 nm	40 K neur. SNN+ANN hybrid	Tsinghua · Nature 2019
Speck (SynSense)	2022	65 nm	327 K neur. DVS on-chip	mW-level · always-on
BrainScaleS-2	2022	65 nm	512 analog neur.	1000× wall-clock

Refs: Merolla 2014 · Davies 2018 · Mayr 2019 · Pei 2019 · SynSense 2022 · BrainScaleS team 2022.

Where SNN Efficiency Comes From

Three structural advantages — none requires a new training scheme.

① Event-driven

No spike → no compute. Inactive neurons skip MAC and memory access entirely.

Typical activity: 5–20 % per timestep

② Multiplication-free

Spike $\in \{0, 1\} \rightarrow W \cdot \text{spike}$ degenerates to W or 0 . Each synapse becomes an Accumulate (AC), not a MAC.

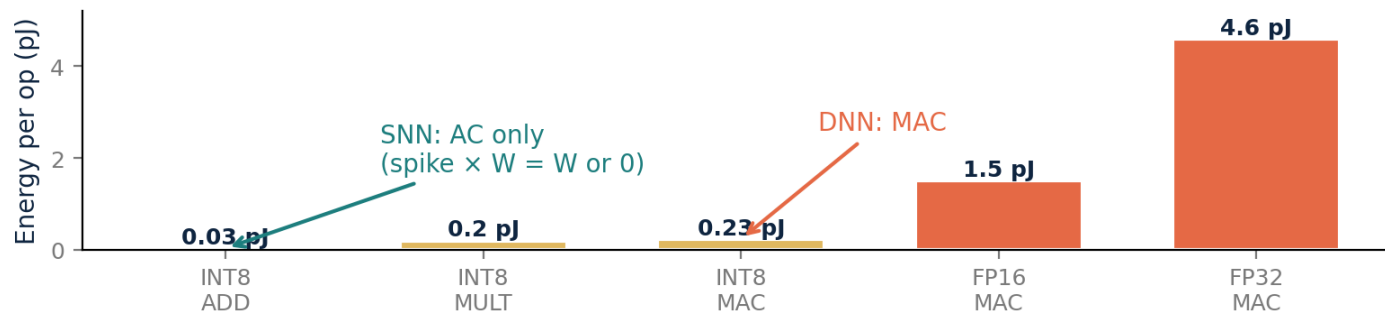
AC $\approx 6\text{--}8\times$ cheaper than MAC (45 nm)

③ Co-located memory

Neuromorphic cores keep synaptic weights in local SRAM, on-chip mesh routing — no DRAM trip.

DRAM 64-bit ≈ 1300 pJ vs SRAM ≈ 5 pJ

Per-operation energy at 45nm [Horowitz, ISSCC 2014]



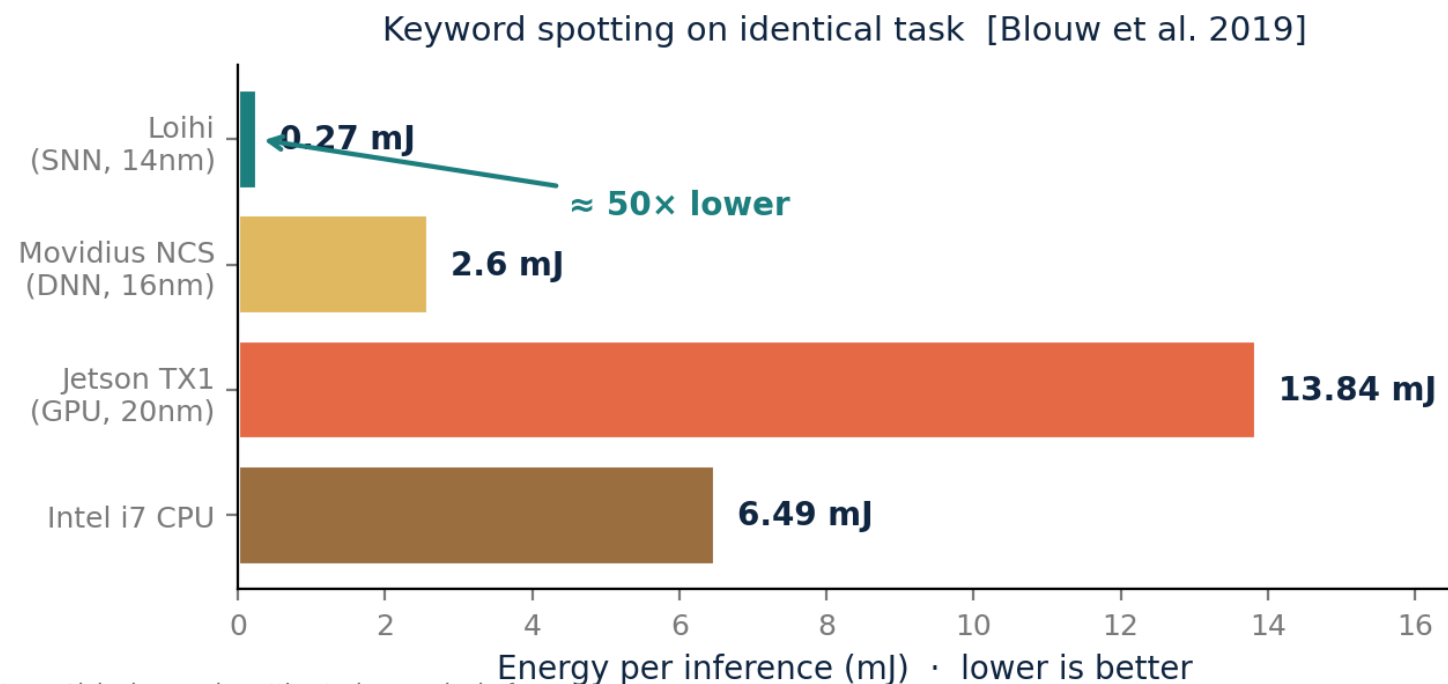
Bottom line

Efficiency is structural — comes from sparsity + binary spikes + co-located memory.

MP-Init / TrSG protect (1) by stabilizing low-T training.

Energy: SNN Chip vs DNN Platforms

Same task, four hardware platforms — what does the gap actually look like?



Same Aloha keyword-spotting task on each platform.

How big is the gap?

Sparse / event tasks

~50–100× vs embedded GPU
(keyword spot, gesture, sparse coding)

Static images (CIFAR / ImageNet)

~3–10× vs GPU at iso-accuracy

Why so task-dependent?

Energy $\propto T \times \text{spike-rate} \times \text{neurons}$.
Low-T + high-sparsity $\rightarrow$ big gap.

Why this matters for our work

MP-Init enables $T = 2$ SOTA $\rightarrow$ directly multiplies energy savings.

What Is an Event-Based Dataset?

Output of asynchronous DVS cameras — sparse, fast, high dynamic range.

DVS: Dynamic Vision Sensor

Each pixel asynchronously emits an event when its log-intensity changes by a threshold.
Output: a stream of (x, y, t, polarity) tuples.

μs

temporal resolution

>120 dB

dynamic range

Sparse

only changes emit

No blur

no exposure window

In this thesis

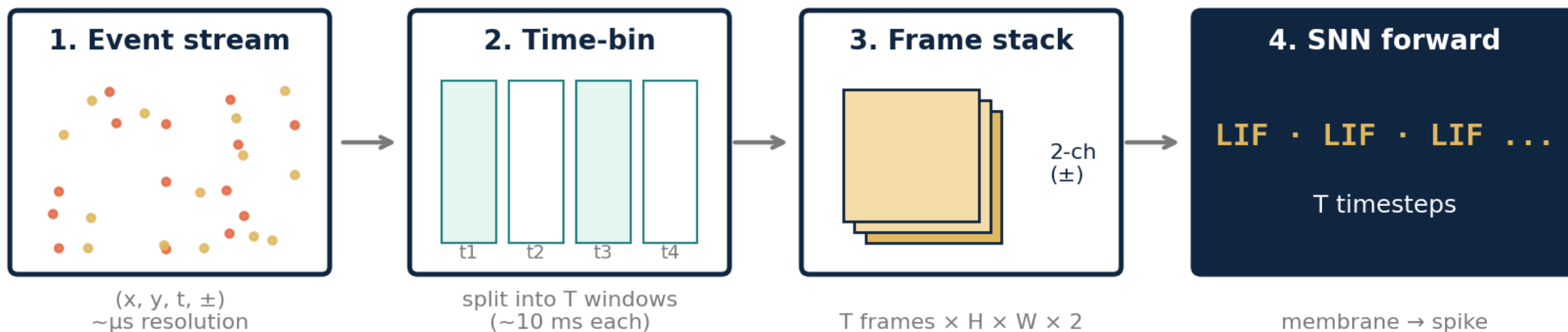
DVS-CIFAR10 (event-based image classification, T = 10).

Dataset	Description	Size
DVS-CIFAR10	CIFAR-10 on monitor + saccading DVS	10 K
N-MNIST	MNIST shown to event camera	70 K
N-Caltech101	Caltech101 in event form	8.7 K
DVS-Gesture (IBM)	11 hand gestures, 29 subjects	1.3 K clips
SHD	Spiking Heidelberg Digits (audio events)	10 K
Gen1 / 1Mpx	Prophesee automotive (object detect.)	39 h / 14 h
ASL-DVS	American Sign Language (event)	100 K

Highlighted: dataset used in this thesis.

From DVS Events to SNN Input

Asynchronous events become T frames that the LIF stack consumes.

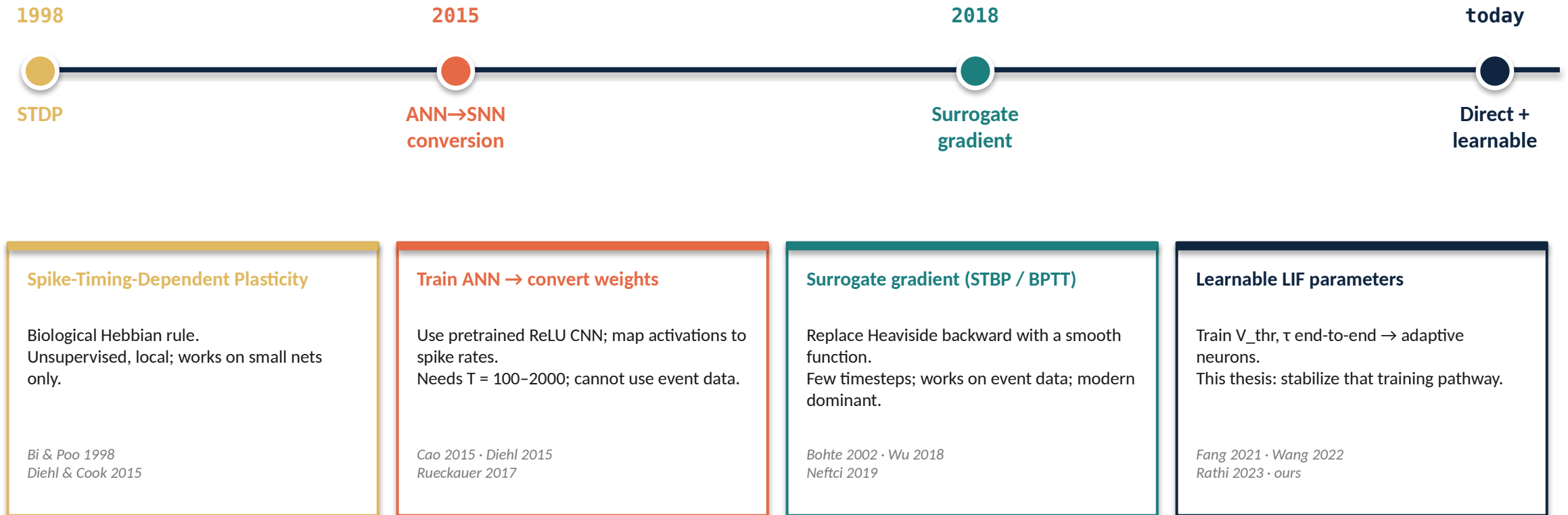


Why this matters for MP-Init

The first frame after binning has no history $\rightarrow$ the membrane starts at zero, far from π . That's exactly the gap MP-Init closes.

How Were SNNs Trained Before?

From biological learning rules to modern surrogate-gradient BPTT.



Where Prior Work Falls Short

TCS methods miss the membrane potential; threshold-learning methods don't diagnose the gradient.

TCS / Normalization in SNNs

Method	What it does	What it misses
BNTT '20	Per-timestep BN params	Output BN only
tdBN '20	Threshold-dependent BN	Output BN only
TEBN '22	Timestep-specific affine	Doesn't fix membrane drift
TAB '24	Temporal-adaptive normalize	Membrane TCS persists

Our take (MP-Init)

Fix the cause, not symptom — align $M[t]$ with stationary π directly.

Trainable LIF parameters (V_{thr} , τ)

Method	What's learned	What it misses
PLIF '21	Leakage τ only	V_{thr} fixed
LTMD '22	$V_{thr} + \tau$ (heavy clipping)	Hides instability
DIET-SNN '23	$V_{thr} + \tau$ (gradient clip)	Doesn't classify failure
Ours (TrSG)	$V_{thr} + \tau$, scale-invariant	—

Our take (TrSG)

Diagnose first (4 failure modes), then cancel the $1/V_{thr}$ factor by construction.